Experiment Documentation

16th January 2026

Contents

This document supplements paper submitted to ISSTA 2026, providing details on the experimental setup and implementation of our study.

1 Environment Setup for Customized Defects4J

The instructions assume a macOS or Linux environment using `bash`. While command syntax may differ slightly between platforms, the overall setup procedure is largely portable across both systems.

1.1 Installing Defects4J

1. Download the customized Defects4J distribution.
2. Add Defects4J to your `PATH` permanently (framework/bin).

1.2 Dependency Configuration

Defects4J depends on several system-level tools and language runtimes. The following instructions describe how to install them on a Linux system.

1.2.1 Linuxbrew

Linuxbrew is a package manager similar to Homebrew on macOS.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.s
```

Follow the instructions printed during installation to finalize the setup.

1.2.2 jq

`jq` is a command-line tool for processing JSON data.

```
brew install jq
```

1.2.3 Java

Install Java 11 or newer and ensure it is available on the command line.

1.2.4 Git

Defects4J requires Git version 1.9 or later.

```
sudo apt update
sudo apt install git -y
```

1.2.5 Subversion

Subversion (svn) version 1.8 or later is required.

```
sudo apt update
sudo apt install subversion -y
```

1.2.6 Perl

Defects4J requires Perl version 5.0.12 or newer.

```
sudo apt update
sudo apt install perl -y
```

cpanm Install `cpanm` (CPAN Minus) after installing Perl:

```
sudo apt update
sudo apt install cpanminus -y
```

Install required Perl packages:

```
sudo cpanm JSON
sudo cpanm --notest JSON::PP XML::Parser::PerlSAX \
  List::Util File::Basename File::Path File::Copy \
  Getopt::Long Time::HiRes String::Interpolate \
  DBI Proc::ProcessTable
```

1.2.7 Python

Our experimental framework requires Python 3 (not Python 2). The following Python packages are needed:

- `numpy`
- `scikit-learn`

1.2.8 Maven

Ensure that Maven is installed and accessible via the command line, as it is required for building Java projects.

1.2.9 Initialize

This is following Defects4j's installation instruction: Initialize Defects4J (download the project repositories and external libraries, which are not included in the git repository for size purposes and to avoid redundancies).

```
cd defects4j
cpanm --installdeps .
./init.sh
```

1.3 Example: Conducting Analysis for a Project Version

This section illustrates the complete analysis workflow for a single Defects4J project version. We provide a wrapper command that prepares all required dependencies, downloads the target project version, and configures the environment by invoking and extending Defects4J's original commands.

Our analysis is performed on the *buggy version* of the program. During the workflow, the program is temporarily patched to the fixed version when validating whether generated test augmentations are able to detect the real bug. We will explain experiment variations and commands in the rest of the documentation.

Project checkout and setup. To prepare a project version, invoke:

```
defects4j get_project <project_id> <version_id>
```

For example, to prepare `Cli-2b`:

```
defects4j get_project Cli 2b
```

This command checks out the project and copies all necessary helper scripts and precomputed coverage information. The list of executed tests is stored in the file `passing_covering_tests`.

After checkout, enter the project directory and start the full analysis:

```
cd Cli_2b
./full_state_analysis.sh
```

For `Cli-2b`, the full analysis takes approximately 13 minutes on our setup using a 2021 MacBook with an M1 Pro chip.

The script encapsulates the full pipeline described below.

Phase 1: Analysis on real bugs. The analysis begins by generating assertion augmentations derived from real bug executions.

- `defects4j states` collects program states from executions on the buggy and fixed versions.
- `defects4j generate_specification` generates assertion specifications from the collected states.
- `java -jar transform.jar` transforms existing tests by injecting assertions derived from the specifications.
- `./verify_all_tests.sh` and `./verify_all_tests_repeat.sh` validate the generated assertions on both the buggy and fixed program versions.

After validation, the script checks whether any generated test reveals the real bug by inspecting `test_outcome.json`. At least one test with outcome `accept` is required for the analysis to continue.

Phase 2: Analysis on surviving mutants. The second phase focuses on assertion generation driven by surviving mutants.

- `defects4j test -c` identifies which tests cover which mutants generated by Major.
- `defects4j mutation_analysis` runs a customized mutation analysis tailored to our experimental setting.
- `defects4j compute_surviving_to_run` selects relevant surviving mutants for state analysis.
- `defects4j states_on_mutants -m -1` collects program states for all surviving mutants. The flag `-1` indicates that states are collected for all surviving mutants; alternatively, a specific mutant ID may be provided.
- `defects4j generate_specification -m` generates assertion specifications from mutant executions.
- `java -jar transform.jar -m` injects mutant-derived assertions into tests.
- `./verify_all_test_mutant.sh` and `./verify_all_test_mutant_repeat.sh` perform repeated execution validation of the mutant-derived assertions.

Finally, each mutant-derived augmentation is evaluated for real-bug detection:

- `defects4j patch -b` restores the buggy version.
- `./verify_detect_bugs.sh` checks whether the mutant-based augmented tests expose the real bug.

Together, these phases fully automate the end-to-end workflow for studying how assertion augmentations derived from real bugs and surviving mutants contribute to real-bug detection.

2 Added and Modified Defects4J Commands Supporting Our Analysis

This subsection documents the Defects4J commands that we added or modified to support our analysis workflow. These commands are specifically customized for our experimental setting and are not part of the standard Defects4J distribution.

Several of these commands depend on artifacts produced by other customized commands (e.g., collected program states, surviving-mutant mappings, or generated oracle specifications). To ensure correct usage, we provide a complete, end-to-end execution workflow in the preceding section Section 1.3, which specifies the required execution order and intermediate outputs.

2.1 defects4j patch

The `defects4j patch` command switches a checked-out project version between its buggy and fixed program states by applying or reverting the developer patch.

The command must be executed inside a valid Defects4J project checkout. Depending on the specified option, it performs one of the following actions:

- `-b`: apply the developer patch and switch the project to the buggy program state;
- `-f`: revert the developer patch and switch the project to the fixed program state.

If the project is already in the requested state (e.g., applying the buggy patch to an already buggy version), the patch operation may block or have no effect. To avoid stalled executions, the command enforces a short timeout: the patch process is automatically terminated after 5 seconds.

This timeout mechanism ensures robustness when repeatedly switching between buggy and fixed program states during large-scale automated analyses.

2.2 Modified `defects4j test` Command

We extend Defects4J's original `defects4j test` command with two additional modes to support our analysis workflow: (i) executing relevant test methods one-by-one, and (ii) executing relevant test methods one-by-one while collecting mutant coverage.

New Options.

- `-e`: identify and execute each relevant developer-written *test method* individually. While Defects4J natively reports relevant *test classes*, this option refines the granularity to the level of individual test methods. The collected test method names are subsequently used by `defects4j coverage -e` (described later) to collect execution coverage on both the buggy and fixed versions of the program.
- `-c`: execute each relevant developer-written test method individually and collect mutant coverage information for each test. This mode enables fine-grained analysis of which mutants are covered by each test method.

2.3 Modified `defects4j coverage` Command

We extend Defects4J's original `defects4j coverage` command with an additional execution mode to support fine-grained coverage analysis at the level of individual test methods.

New Option.

- `-e`: execute each relevant developer-written test method individually and collect coverage information for each test. In contrast to Defects4J's default behavior, which reports coverage at the test-suite level, this option enables per-test coverage analysis on both the buggy and fixed versions of the program. Specifically, for each executed test method, the coverage report and terminal log record the exercised code and the classes loaded during execution.

2.4 defects4j states

The `defects4j states` command collects program states for a checked-out Defects4J project version by executing selected test methods under state instrumentation.

This command is designed to support state-based analysis of real bugs by systematically capturing output states on both buggy and fixed versions of the program.

Usage.

```
defects4j states
```

Behavior. The command expects a file named `passing_covering_tests` in the current working directory, containing one test method per line. For each listed test method, the command performs the following steps:

1. Compile the project tests and instrument the program for state collection.
2. Execute each test method on the buggy version of the program and record the resulting program states.
3. Switch to the fixed version using `defects4j patch -f`, recompile and re-instrument the program, and execute each test method twice to collect program states.
4. Restore the project to the buggy version using `defects4j patch -b`.

Outputs. Collected program states are written to the directory `all_states/`, organized by test method and program version. Specifically, for each test method:

- states from the buggy version are stored under `all_states/<test_method>/buggy/state.json`;
- states from the fixed version are stored under `all_states/<test_method>/fixed/{1,2}.json`.

If any tests fail during state-instrumented execution, the failing test names are recorded in `failing_with_instrumentation.txt`.

2.5 defects4j generate_specification

The `defects4j generate_specification` command generates assertion specifications from previously collected program states. These specifications are later used to synthesize assertion-based test augmentations.

Usage.

```
defects4j generate_specification
defects4j generate_specification -m
```

Modes. The command supports two execution modes, depending on whether specifications are generated from real-bug executions or from mutant executions.

- **Bug-based specification generation (default).** When invoked without additional options, the command generates assertion specifications from program states collected on real bugs. This mode consumes the state traces stored in `all_states/` and produces oracle specifications that characterize behavioral differences between the buggy and fixed versions of the program.
- **Mutant-based specification generation (`-m`).** When invoked with the `-m` option, the command generates assertion specifications from program states collected for surviving mutants. In this mode, the analysis leverages both `all_states/` (real-bug executions) and `mutant_states/` (mutant executions) to derive mutant-specific oracle specifications.

Outputs. Depending on the selected mode, the generated specifications are written to:

- `oracle_specification/` for bug-based specification generation;
- `mutant_oracle_specification/` for mutant-based specification generation.

Role in the pipeline. The generated assertion specifications serve as inputs to subsequent test transformation steps, where assertions are injected into existing tests to enable targeted detection of real bugs or surviving mutants.

2.6 defects4j mutation_analysis

The `defects4j mutation_analysis` command performs a customized mutation analysis designed specifically for our experimental workflow. This command is *not* equivalent to Defects4J's original `defects4j mutation` command.

Behavior. The command iterates over all generated mutants and, for each mutant, performs the following steps:

1. Configure the mutation by dynamically updating the active mutant identifier in the mutation configuration.
2. Compile and install the mutated bytecode into the project build directory.
3. Execute relevant developer-written tests to determine whether the mutant is killed or survives.
4. Filter failing tests to exclude failures that already occur on the fixed version or correspond to known triggering tests, ensuring that only mutant-specific failures are considered.
5. Record the mutant outcome and execution statistics.

Timeout and fail-fast policy. Mutant execution is guarded by a timeout mechanism. Each mutant is executed with a timeout of `180` seconds. If test execution exceeds this limit, the mutant execution is forcibly terminated and the mutant is treated as killed.

The value of 180 seconds is a conservative and safe threshold for our experimental environment (MacBook Pro 2021 with an M1 Pro chip), effectively identifying mutants that lead to infinite

loops or excessively long executions. While we customize timeout values for specific subjects during development, a 180-second timeout is sufficient and safe across all subjects used in our experiments.

In addition to the timeout, the command employs fail-fast termination based on early detection of mutant-specific test failures and resource usage constraints, further reducing unnecessary execution overhead.

Output. The command produces a JSON summary file, `full_mutation_analysis.json`, which records the number of killed and surviving mutants, the overall mutation score, and the identifiers of killed and surviving mutants.

2.7 defects4j compute_surviving_to_run

The `defects4j compute_surviving_to_run` command computes an optimized execution plan for surviving mutants by determining, for each mutant, the subset of test methods that should be executed in subsequent analysis steps.

This command is part of our customized mutation-driven analysis pipeline and is used to reduce redundant test executions when collecting program states for surviving mutants.

Purpose. After mutation analysis, only a subset of mutants survive execution. Moreover, not all tests cover every surviving mutant. This command identifies, for each surviving mutant, the relevant test methods that both: (i) cover the mutant, and (ii) appear in the oracle-specification set used by our analysis. The resulting mapping enables targeted and efficient downstream execution.

Behavior. The command performs the following steps:

1. Read the list of surviving mutant identifiers from `full_mutation_analysis.json`.
2. Extract the set of valid test methods by traversing all JSON files under `oracle specification/` and collecting unique `test_name` entries.
3. Load per-test mutant coverage information from `mutant_coverage.json`.
4. For each surviving mutant, retain only those test methods that both cover the mutant and appear in the extracted test set.
5. Discard surviving mutants for which no such test methods exist.

Output. The command produces a JSON file named `surviving.json`. This file maps each surviving mutant identifier to the list of test methods that should be executed for that mutant in subsequent state-collection and specification-generation steps.

2.8 defects4j states_on_mutants

The `defects4j states_on_mutants` command collects execution states for *surviving mutants* by executing only the test methods that are relevant to each mutant. This commands is similar to `defects4j states` which collects states on real bugs.

Purpose. This command performs *targeted state collection* by running, for each surviving mutant, only those test methods that (i) cover the mutant and (ii) are required by our oracle-specification analysis. The collected states form the basis for subsequent assertion and oracle construction.

Usage. The command supports two execution modes via the `-m` option:

- `-m <mutant_id>`: collect states for a single surviving mutant;
- `-m -1`: collect states for *all* surviving mutants listed in `surviving.json`.

Output. Execution states are written to the directory `mutant_states/`, organized by mutant identifier and test method. For a mutant `m` and test method `t`, the corresponding state is stored at:

`mutant_states/m/t/state.json`.

2.9 defects4j update_oracle_helper

We add a small helper command, `defects4j update_oracle_helper`, to prepare the oracle-validation script used in our workflow. Depending on whether we are executing generated assertions on real bugs or surviving mutants, the command installs the corresponding helper implementation into the current project directory.

```
# install helper for real-bug analysis
defects4j update_oracle_helper -b

# install helper for mutant analysis
defects4j update_oracle_helper -m
```

Behavior. The command copies the appropriate helper script from the Defects4J installation directory into the current working directory under a unified name: `verifyOracleData.py`. Specifically, `-b` installs `verifyOracleData_Bug.py`, whereas `-m` installs `verifyOracleData_Mutant.py`.

Output. After execution, the working directory contains:

- `verifyOracleData.py`: the oracle-validation helper used by the subsequent verification scripts in our pipeline.

3 Coverage Collection

3.1 Coverage for Real Bugs

As described in the paper, to support coverage analysis involving real bugs, we first identify *bug-impacted lines* from the diff hunks between the buggy and fixed program versions. A test method is considered to *execute the real bug* if it satisfies any of the following conditions when executed on either the buggy or the fixed program version:

1. it executes at least one bug-impacted line;
2. it instantiates a bug-impacted class when the bug modifies instance fields; or
3. it loads a bug-impacted class when the bug modifies static fields or when classes are added or removed.

We use Cobertura for coverage collection, which is integrated into Defects4J by default. Specifically, we use the customized command `defects4j coverage -e` to collect coverage information for each developer-written test method individually. For each test execution on both the buggy

and fixed program versions, we record (i) the executed source lines and (ii) the set of classes loaded by the JVM during the test execution.

Whether a bug affects instance fields, static fields, or involves class-level changes is determined via manual inspection and labeling of each bug patch. Based on the collected coverage information, we identify the set of *passing covering test methods*, which are written to the file `passing_covering_tests`. This set is later used to restrict state collection and assertion augmentation to tests that are relevant to the real bug.

Handling Coverage Failures. We note that coverage collection may fail for a small number of bugs due to known issues in Defects4J and Cobertura. For example, coverage fails for JSOUP bugs 4, 6, and 9, as documented in the Defects4J issue tracker. [1] In such cases, we conservatively treat *all* developer-written test methods as covering tests. Since coverage is used solely as an optimization to reduce the cost of state collection—and not as part of the core analysis—this fallback does not affect the correctness of our results. When coverage fails, we simply perform state analysis on all passing tests.

3.2 Coverage for Mutants

For mutants, coverage information is used to determine which test methods potentially execute each surviving mutant. This capability is enabled by Major, which allows individual mutants to be activated and executed by specifying their mutant identifiers at runtime.

Although Major is closed-source, we found that the runtime behavior relevant to our analysis is controlled by a single configuration file, `Config.java`, contained in `major-rt.jar`. We reverse-engineered this behavior from the compiled class file and extracted the corresponding source-level configuration, which we provide in `mutation_testing_utils/Config.java`.

Specifically, setting `__M_NO = 0` enables mutant coverage collection, whereas setting `__M_NO > 0` activates a specific mutant. In the static initialization blocks of this configuration file, we attach a custom hook that records the identifiers of all mutants that are potentially executed by a given test method. These mutant identifiers are collected and written to disk before the JVM terminates.

The resulting mutant-to-test coverage information is later used to restrict state collection to relevant test–mutant pairs, substantially reducing the overall analysis cost while preserving completeness.

4 Assertion Generation

4.1 Test Code Instrumentation

To enable assertion generation, we instrument *test code* to monitor and record runtime program states observed during test execution. The instrumentation is implemented at the JVM bytecode level using ASM. The implementation is available in the `observerForSpecification` module, and is distributed as a standalone JAR (`o.jar`) that can be directly integrated into our analysis pipeline.

Conceptually, the instrumentor transforms each test method into a monitored execution region and injects lightweight hooks at key *state-observation sites* (e.g., method returns, local variable stores, and field reads). These hooks forward values—together with precise source identifiers—to a runtime collector (`org/helper/InstrumentationUtils`) which normalizes values and logs them for later assertion synthesis.

[1] `https://github.com/rjust/defects4j/issues/627`

Instrumentation target: test methods only. We instrument only methods that are considered test methods under common JUnit conventions: (i) JUnit 3-style methods whose names start with `test` in classes that extend `junit.framework.TestCase`, and (ii) JUnit 4/5-style methods annotated with `@Test`. To detect JUnit 4/5 tests, we track annotations at the bytecode level via `visitAnnotation` and set a flag when observing `Lorg/junit/Test;` or `Lorg/junit/jupiter/api/Test;`. All other methods are left unchanged.

Test-level lifecycle hooks. For each instrumented test method, we inject two lifecycle callbacks: `TesExtension.onTestMethodStart()` at method entry and `TesExtension.onTestMethodEnd()` at every exit (both normal return and exceptional exit). We implement the exceptional-path handling by surrounding the method body with a synthetic try/finally pattern: we mark the protected region (`startL`–`endL`), install a catch-all handler for `Throwable`, invoke the end hook in the handler, and rethrow the exception. This ensures the end hook is executed regardless of how the test terminates, which is critical for delimiting per-test traces.

Source locations and site identity. We use bytecode line-number tables (via `visitLineNumber`) to associate each observation site with a source line. Each recorded value is tagged with: (1) a *location id* `fileLoc-lineNumber`, where `fileLoc` is the internal class name visited by ASM (e.g., `org/foo/TestClass`) and `lineNumber` is the most recently observed line number; (2) an *event label* describing the kind of site and its static signature (e.g., `return-owner-name-desc`, `local-desc-varName`, `getField-owner-name-desc`, `array-type`); and (3) an *ordinal* that disambiguates repeated executions of the same syntactic site (e.g., within loops or multiple invocations on the same line). Ordinals are generated by per-site counters keyed by (file, line, and static signature), ensuring each event instance is uniquely identifiable within a test trace.

Instrumented state-observation sites. Within each instrumented test method, we inject hooks at the following bytecode events:

1. **Method-call return values.** After each method invocation (`visitMethodInsn`) that produces a non-`void` value, we intercept the value on the operand stack and forward it to `InstrumentationUtils`. We skip instrumentation for `void` returns and certain constructor-related cases to avoid interfering with object initialization semantics. For primitive and `String` return types, the hook returns a (possibly normalized) value of the same type; for reference types, the hook observes the object (by duplicating the reference with `DUP`) and records it without altering the stack value.

2. **Local variable stores.** On each local store instruction (`ISTORE`, `LSTORE`, `FSTORE`, `DSTORE`, `ASTORE`) we instrument the stored value. We rely on a pre-computed list of local-variable metadata (`VariableTrackingClassVisitor.result`) that provides the static descriptor and source name for each store site. Similar to return values, primitives and `String` are passed through type-specific hooks, while reference types are recorded via an observation hook that does not change the stored value (using `DUP` before the call).

3. **Field reads.** For field reads (`GETFIELD` and `GETSTATIC`), we instrument the loaded field value after the field instruction executes. We ignore reads whose owner belongs to `org.assertj` to avoid perturbing assertion-library internals and to reduce noise. Field-read events are tagged with `getField-owner-name-desc` and recorded through the same type-specific processing hooks.

4. **Array allocations.** We instrument newly allocated arrays to capture array references that may later be compared, indexed, or passed across API boundaries. Specifically, we instrument: (i) object arrays created by `ANEWARRAY` (`visitTypeInsn`), (ii) primitive arrays cre-

ated by NEWARRAY (visitIntInsn) with the element kind inferred from the operand (T_INT, T_BOOLEAN, etc.), and (iii) multi-dimensional arrays created by MULTIANEWARRAY. After allocation, we duplicate the array reference and record it as an object event with an array-... label.

Type-specific processing hooks. To preserve bytecode correctness, we dispatch to a type-specialized hook based on the static type at the instrumentation point. For primitives we use processInteger, processLong, processDouble, processFloat, processByte, processChar, processShort, and processBoolean; for strings we use processString; and for all other reference types we use processObject. Each hook receives: the observed value (or reference), the location id (fileLoc-lineNumber), an event label, and an ordinal. Primitive and string hooks return the same type to keep the operand stack consistent; object hooks record observations without mutating the reference value.

Line-number fidelity. Our line-number mapping is derived from compiler-emitted line tables and thus inherits known limitations: multiple source statements may map to a single line entry; loops, lambdas, and inlined expressions may produce discontinuous or coarse mappings; and classes compiled without debug line information may have missing or synthetic line numbers. We nevertheless found line-level attribution sufficiently precise for grouping observations by program point during assertion synthesis.

Summary. Overall, the instrumentation turns each test method into a trace-producing execution that is (i) delimited by reliable start/end hooks, (ii) restricted to test methods, (iii) optionally bounded by a domain scope, and (iv) enriched with uniquely identified runtime observations at returns, local stores, field reads, and array allocations. These traces form the raw evidence used by our assertion generator to infer candidate assertions at relevant program points.

4.2 State Representation

To synthesize assertions from observed runtime values, we must convert arbitrary Java objects into a stable, structured representation. The core utilities for state representation are implemented under state_utils/helper (notably org.helper.graph). At runtime, each observed value is serialized into a bounded *object graph* that captures its type, internal structure, and recursively reachable sub-objects up to a configurable depth. Their states are dumped in the json format.

Graph-based serialization. Given an observed value o and a depth bound d, the serializer constructs a directed graph rooted at o using a breadth-first traversal. Each node is one of four variants: (i) Leaf for simple values, (ii) VNormal for ordinary objects represented by their fields, (iii) VCollection for arrays and iterables represented by their elements, and (iv) VMap for maps represented as key–value pairs. The root node is labeled root, and each node records (a) a logical name (e.g., field name, element, key, value), (b) a runtime type string, and (c) its traversal depth.

Base cases and normalization. The serializer treats primitive wrappers, String, and null as *simple* objects and represents them as Leaf nodes carrying their stringified values. For JDK library objects whose runtime types begin with java., javax., or sun., we avoid reflective expansion and instead record a normalized toString() value. In particular, we strip trailing identity-hash suffixes (e.g., @7a3f1c) to reduce flakiness stemming from address-like identifiers.

Collections, arrays, and maps. For collections and arrays, we represent each element as a child node under a `VCollection` node. Primitive arrays are serialized as a single `Leaf` using `Arrays.toString( )`; non-primitive arrays and iterables are expanded element-wise. For sets and maps, we attempt to deterministically order entries by sorting keys (or elements) when they are comparable; if sorting fails, we fall back to the iteration order. Maps are represented as a `VMap` node whose children are explicit key/value subgraphs.

Ordinary objects via reflective field expansion. For non-JDK objects that are neither maps nor collections, we use reflection to enumerate declared fields across the class hierarchy. We exclude synthetic/hidden fields (names starting with `$`), transient fields, and *unmutable* fields (final primitives or final strings), as these provide limited diagnostic value and may unnecessarily increase trace size. Remaining fields are sorted by field name to yield a stable ordering. Each field value is then represented recursively as either a `Leaf` (for simple values) or a structured node (`VNormal`, `VCollection`, `VMap`) for compound objects.

Cycle handling and depth bounding. To prevent infinite recursion and excessive overhead, the serializer enforces two safeguards. First, it maintains a reference-based `visited` set and replaces repeated references with a `Leaf` node whose value is `visited`. Second, it bounds expansion by depth: nodes deeper than d are not further expanded. As a result, the representation is finite and robust even in the presence of cyclic object graphs.

Value sanitization. When converting leaf values to strings, we apply conservative sanitization to avoid recording unstable or environment-specific data. In particular, we (i) truncate extremely long strings, (ii) mask multi-line strings with indentation to preserve readability, and (iii) discard "suspicious" strings that resemble object addresses, mocks, or synthetic lambda identifiers. This reduces the risk that generated assertions depend on nondeterministic or platform-specific artifacts.

Additionally, we apply specialized handling for `double`-typed values during state comparison to ensure robust and stable equality checks.

4.3 Assertion Specification Generation

After collecting runtime states for each test execution (as described in Sections 4.1 and 4.2), we generate an *assertion specification* that precisely identifies *where* and *what* to assert. Intuitively, an assertion specification is a structured descriptor of a *candidate observation point* (e.g., a return value, a field read, or a local variable store) together with a *path* to the concrete subvalue inside the serialized state that differs across program versions. The core implementation is provided in `state_utils/python_scripts/generate_assertion_specification.py`.

Inputs and outputs. For each test case, our state collector produces a JSON trace (e.g., `state.json`) that contains: (i) a list of metadata entries (`metas`) describing each instrumented observation site, and (ii) a graph-structured value representation in which shared objects are encoded using JSON references (`$id`/`$ref`). Given these traces, the specification generator emits a set of per-assertion JSON files (e.g., `assertion_#`.json) under the `oracle specification` directory, each describing one candidate assertion to be materialized in test code.

Resolving shared references and reconstructing values. Because object graphs may contain sharing and cycles, the trace uses `$id`/`$ref` to represent repeated references. The generator first builds a lookup table for all `$id` nodes and then resolves `$ref` pointers to their targets. To enable robust structural comparison, we reconstruct the resolved JSON into nested

containers (maps, lists, sets) wrapped by a lightweight `NodeWrapper` abstraction that supports deep equality and hashing based on *structure and values* (ignoring node identifiers).

Detecting behavioral differences. For each test, we compare the serialized states between the buggy and fixed versions and identify the set of *differing nodes*. Concretely, we perform a deep recursive diff over the reconstructed graphs and collect the `$id` (or a fallback index) of any node whose structure or leaf value differs. To reduce false positives due to nondeterminism, we also compute a stability mask by diffing two independent fixed-version runs; any node that differs across fixed runs is treated as unstable and excluded from candidate generation.

Floating-point normalization in comparisons. During diffing, leaf values are compared as strings with special handling for numeric literals. In particular, we normalize floating-point values (including scientific notation) into a canonical textual form (e.g., fixed precision, normalized zero) before comparison. This reduces spurious differences caused by floating-point formatting artifacts (e.g., `-0.0` vs. `0.0` or minor textual variation).

Mapping differing nodes to access paths. Each `$id` in the state graph is associated with two complementary paths: (i) a *human-readable* path (e.g., `var.foo.bar[key].baz`) used for reporting and debugging, and (ii) a *programmatic* access path represented as a list of JSON keys/indices used to retrieve the value from the serialized trace deterministically. We compute these paths by traversing the JSON structure and recording, for each `$id`, both its display path and its concrete access sequence (including `entries`/`elements` indices for maps and collections).

Binding a difference to an instrumentation site. Each observation site is described by a corresponding entry in `metas`, which includes attributes such as: the site kind (`return`, `getField`, `local`), the owning type, member name, return type (if applicable), source file/line identifier, and an ordinal to disambiguate repeated executions. For each differing node, we locate the associated `metas` entry and construct a site identifier by concatenating these attributes. When a site identifier occurs multiple times within a test trace (e.g., due to loops), we track the occurrence count and cap candidates to the first few iterations to avoid generating assertions for deep loop iterations.

Filtering and candidate selection. We apply several conservative filters to keep only actionable and stable candidates: (1) we skip specifications derived from `static` observation sites; (2) we require that the observation originates from the current test class file (excluding helper methods in other files); (3) we drop candidates whose access paths are too deep or that index into very large collections (to avoid brittle assertions and excessive synthesis cost); and (4) we drop unstable nodes identified by the fixed-versus-fixed stability mask. Currently, we generate specifications for `return`, `getField`, and `local` sites; array-allocation sites are recorded in traces but not yet materialized into assertion specifications.

Specification schema. Each emitted assertion specification records enough information to later synthesize a concrete assertion statement at the correct location. At a high level, each JSON specification includes: the observation `source` (`return`/`getField`/`local`), the owning type and member/variable name, the site ordinal, the test identifier (`test_name`) and line number in the test file, and the selected access path to the differing value (`python_access`) along with a readable counterpart (`readable_access`). These specifications serve as the contract between (i) state differencing and (ii) assertion materialization: the former decides *what to assert*, and the latter decides *how to express it* in the target assertion framework.

An example of the assertion specification schema is provided in the `oracle specification` directory.

4.4 Source-Code Instrumentation for Assertion Materialization

After generating assertion specifications (Section 4.3), we *materialize* each specification by rewriting the corresponding test source file to expose the target runtime value and record it for assertion synthesis. This stage operates at the Java *source-code* level (rather than bytecode) and is implemented using JavaParser with symbol resolution. The implementation is located in the `SourceCodeInstrumentation` folder.

Goal. Given an assertion specification that identifies (i) an observation site (return value, field read, or local variable) and (ii) a stable access path into the serialized state, our rewriter injects a lightweight probe `org.helper.Assertions.verify(key, value)` into the test method. At runtime, `verify` records the concrete value associated with the specification key, which is later used to synthesize an assertion statement.

Inputs and outputs. The input to this phase is a directory of per-assertion JSON specifications (e.g., `oracle specification` for real bugs and `mutant_oracle_specification` for mutants). For each specification, the instrumentor: (1) locates the target test file on disk, (2) parses it into an AST, (3) rewrites the AST to expose the target value and add `verify` calls, and (4) emits an instrumented test source file. For traceability and debugging, we also save the original test file snapshot ("before") alongside the rewritten version ("after").

Specification parsing and dispatch. Each specification is parsed into a strongly-typed `Spec` object (e.g., `source`, `name`, `owner`, `returnType`, `line_number`, `ordinal`, and an optional `loop` index). Based on `Spec.source`, the instrumentor dispatches to one of four rewriters: (1) `return` sites: `ChainCaptureRewriter`, (2) constructor returns: `ConstructorCaptureRewriter`, (3) `getField` sites: `FieldCaptureRewriter`, and (4) `local` sites: `LocalCaptureRewriter`. Each rewriter targets a specific AST pattern and transforms it into an explicit temporary variable followed by one or more `verify` calls.

Locating the target expression. We identify the target expression using a combination of (i) source line constraints and (ii) an ordinal (Nth occurrence) recorded during bytecode instrumentation. For method calls and field accesses, the instrumentor collects candidate AST nodes within the target line range, orders them to approximate left-to-right textual order, and then selects the Nth candidate. To improve robustness under minor line-number mismatches (a known limitation when mapping bytecode line tables back to source), the rewriters may search within a bounded window around the target line (e.g., scanning downward for constructors or upward for locals).

Placement of `verify` probes (assertions. To ensure the recorded value reflects the method execution path, we insert `verify` calls: (1) immediately before each `return` statement in the enclosing test method, and (2) additionally at the end of the method body if no such call exists (to cover methods with no explicit return). For tests that expect exceptions (e.g., JUnit `@Test(expected=...)`), we conservatively place `verify` earlier (using a heuristic insertion point derived from the fixed-state trace) to avoid missing the probe due to early termination.

Loop-sensitive capture. If the specification indicates the observation occurred inside a loop, the instrumentor records a loop iteration index and uses a small guard to capture the value from the target iteration. Specifically, we introduce an iteration counter (`_assert_counter_`) and a

dedicated capture variable (`_assert_var_`), and assign `_assert_var_` when the counter matches the desired iteration. Subsequent `verify` calls use `_assert_var_` to avoid conflating values across iterations.

Type handling and symbol resolution. To generate compilable source code, temporary variables are declared with either the provided type from the specification (preferred) or a safe fallback (e.g., `var`). We configure JavaParser with a combined symbol solver that includes the JDK and project/test source roots (when present), enabling us to filter and match candidates based on receiver type and return type when resolution succeeds. If resolution fails (e.g., missing classpath entries), we fall back to syntactic matching to avoid dropping candidates unnecessarily.

4.5 Expected Value and Runtime Assertion Behavior

Our generated assertions are *state-based*: rather than hard-coding a literal expected value into the test source, each injected assertion dynamically serializes the runtime value of a target expression and compares it against the *expected state* recorded from the *fixed* program version. This design decouples assertion checking from syntactic equality of Java values and enables structured comparison of nested objects, collections, and maps.

Runtime assertion entry point. All injected checks call a single runtime hook,

`org.helper.Assertions.verify(String message, Object obj)`.

At runtime, `verify` performs three steps: (1) it serializes the observed object `obj` into a JSON state representation; (2) it locates the corresponding expected state in the pre-recorded fixed-version state file using the assertion specification indexed by `message`; and (3) it compares the two states and throws an `AssertionError` when a mismatch is observed.

Step 1: serializing the observed value. When `verify` assertion is executed, it wraps the observed runtime value into a `StateItem` and exports it using our JSON serializer (`TesExporter.exportVar`). The serializer traverses the object graph to a bounded depth (to avoid excessive overhead) and emits a structured JSON representation that preserves container structure (e.g., maps, iterables, arrays) and fields of user-defined objects. The resulting JSON is written to a temporary file at `stateData/temp/statefile.json`, which serves as the *actual* observation for the current assertion evaluation.

Step 2: retrieving the expected value via the specification. Each assertion is associated with a unique *specification file* (Section **??**) that records where the expected value resides in the fixed-version state dump. At runtime, `verify` parses `message` to recover the specification index:

$$\texttt{oracle\_index} \leftarrow \texttt{message.split("\_")[1]}\ .$$

It then calls a Python checker, `verifyOracleData.py`, passing this index. The checker loads: (i) the corresponding specification JSON (from the `oracle specification` directory), (ii) the pre-recorded fixed state file for that test (from `all_states/<test>/fixed/1.json`), and (iii) the freshly dumped actual state file (from `stateData/temp/statefile.json`).

The specification provides a `python_access` path (a list of keys/indices) that deterministically selects the expected JSON node in the fixed state file. In parallel, the checker navigates into the actual runtime dump to the corresponding node (typically under `vardata["metas"][0]["graph"]`, followed by the suffix of the same access path).

Step 3: state comparison and failure semantics. After extracting the expected node (fixed-version) and the actual node (current execution), `verifyOracleData.py` compares them structurally. To handle sharing and potential cycles in serialized graphs, our JSON representation uses `$id`/`$ref`. Before comparison, the checker resolves references and compares object subgraphs by recursively diffing nested structures. Concretely, the checker computes a set of differing node identifiers via a deep structural diff (`deep_diff_ids`); if the diff is empty, the assertion is considered satisfied.

If the checker reports a mismatch, `Assertions.verify` throws `AssertionError("assertion failed!")` and the corresponding test fails.

Interpretation of the "expected value." In our framework, an "expected value" is not restricted to a single primitive literal; rather, it is a *projection* of the fixed-version runtime state at a particular access path, which may span nested objects and container elements, although in most cases it reduces to a literal value. This representation captures both value changes (e.g., field values) and structural changes (e.g., missing keys or different collection shapes), subject to the serializer depth bound and our filtering heuristics described in Section 4.7.

4.6 Execution Validation

To mitigate the impact of non-deterministic behavior, we perform repeated execution validation for all generated assertions. Specifically, each assertion is executed at least four times: two executions on the original (passing) program version and two executions on the faulty or mutant version, where the assertion is expected to fail. This baseline validation procedure is fully automated via the script `full_state_analysis.sh`.

Beyond this minimum, we apply additional rounds of execution validation whenever non-determinism is detected. Concretely, if *any* generated assertion exhibits inconsistent outcomes across repeated executions—either failing intermittently on the fixed program or passing intermittently on the faulty program—we classify the behavior as flaky and trigger further validation runs. This process is repeated for the entire pool of generated assertions for a project until no such flakiness is observed.

In practice, we find that non-deterministic behavior is largely concentrated in a small number of subjects, most notably JacksonDatabind and JacksonCore.

4.7 Limitations and Experimental Setup

This section summarizes limitations of our instrumentation pipeline and the guardrails we adopt in our experiments to keep the generated assertions reliable and the end-to-end workflow tractable.

Line-number mismatches between bytecode and source. Our observation sites are first identified at the JVM bytecode level (ASM) and later materialized by rewriting Java source code (JavaParser). However, line numbers in bytecode debug tables do not always align perfectly with Java source statements (e.g., compiler folding, synthetic code, multi-line expressions). As a result, JavaParser may fail to locate the intended AST node at the exact line. To mitigate this, our source rewriters employ an *approximate matching* fallback: when an exact match is unavailable, we search within a bounded window around the target line and then select the Nth candidate consistent with the recorded ordinal.

Instrumentation may perturb program behavior. Although the instrumented tests are required to compile and pass, our pipeline injects additional code and performs state serializa-

tion. State dumping (e.g., graph construction, reflective field access, and JSON emission) can perturb timing, memory pressure, exception ordering, and other subtle behaviors. Thus, it is possible for the instrumentation to alter program behavior even when tests still pass, which may affect the representativeness of the captured states and the resulting assertions.

Limits of mapping bytecode observations to source constructs. Certain bytecode patterns do not have a clean one-to-one correspondence with high-level language features in the original source code (e.g., compiler-generated temporaries, desugared constructs, synthetic accessors, or inlined expressions). Consequently, not all bytecode-level observation sites can be faithfully mapped back to a stable and editable source location. This limitation is inherent to the cross-representation nature of our pipeline.

Loop-related assertions. Assertions inside loops are challenging because the observed value may differ across iterations. We support loop-indexed capture via a counter-based mechanism, but this approach may still be brittle for complex loop bodies (e.g., early exits, nested loops, iterator invalidation, or non-deterministic iteration order). To reduce flakiness, we impose a conservative bound: if the loop index recorded in the specification exceeds 10, we skip that assertion candidate.

Unsupported test styles and unreliable cases. We intentionally exclude several classes of tests and situations where assertion synthesis is unlikely to be reliable: (1) **Trace-changing executions.** If the execution trace differs across runs (e.g., between two "fixed" executions), we do not generate assertions for that test, since the observation site is unstable. (2) **Expected-exception tests.** Assertions generated for tests using JUnit's `@Test(expected=...)` are not reliable due to early termination and altered control flow; we treat these as unsupported. (3) **Parameterized tests.** We do not generate assertions for parameterized tests, as the shared method body is executed under multiple inputs and a single injected assertion may not generalize across parameters. (4) **Non-`@Test` method bodies.** We do not inject assertions outside the body of the `@Test`-annotated method (e.g., shared setup/helpers), since such code is reused across many tests and would conflate multiple test contexts.

Resource caps and filtering heuristics. To keep the pipeline scalable and avoid extreme state explosion, we apply conservative bounds during specification generation and materialization: (1) we only generate assertions for state files of at most 10 MB; (2) we skip assertion paths that require accessing deeply nested structures or overly long paths, including: (i) access paths that exceed a depth threshold (5), (ii) collection/array element accesses beyond the first 10 elements, (iii) variable indices beyond the first 1000 recorded variables; (3) we skip assertion sites with loop index greater than 10, as noted above. These caps trade completeness for tractability and robustness.

Tooling constraints: coverage and mutation analysis. Our pipeline interacts with Defects4J/Major and coverage tooling, which introduces additional experimental constraints. First, Java coverage scans are not always accurate at the granularity of individual test methods, and some test suites cannot be executed reliably on a per-test basis (i.e., they must be run as a whole). We therefore manually identify such cases and treat them specially in our execution harness. Second, coverage collection in the presence of Major is not always reliable and may fail to run for certain projects due to tool incompatibilities. Finally, we enforce a subject-specific timeout threshold when running Major to bound execution cost; the threshold differs across subjects due to their heterogeneous build systems and test-suite sizes.

Type-system limitations. Our technique does not handle Java generics robustly in all cases. In particular, type resolution may be incomplete when symbol solving fails or when generic instantiations are erased or represented ambiguously. This can lead to conservative filtering, reduced matching accuracy for target expressions, or skipped candidates.